

13. JDBC编程

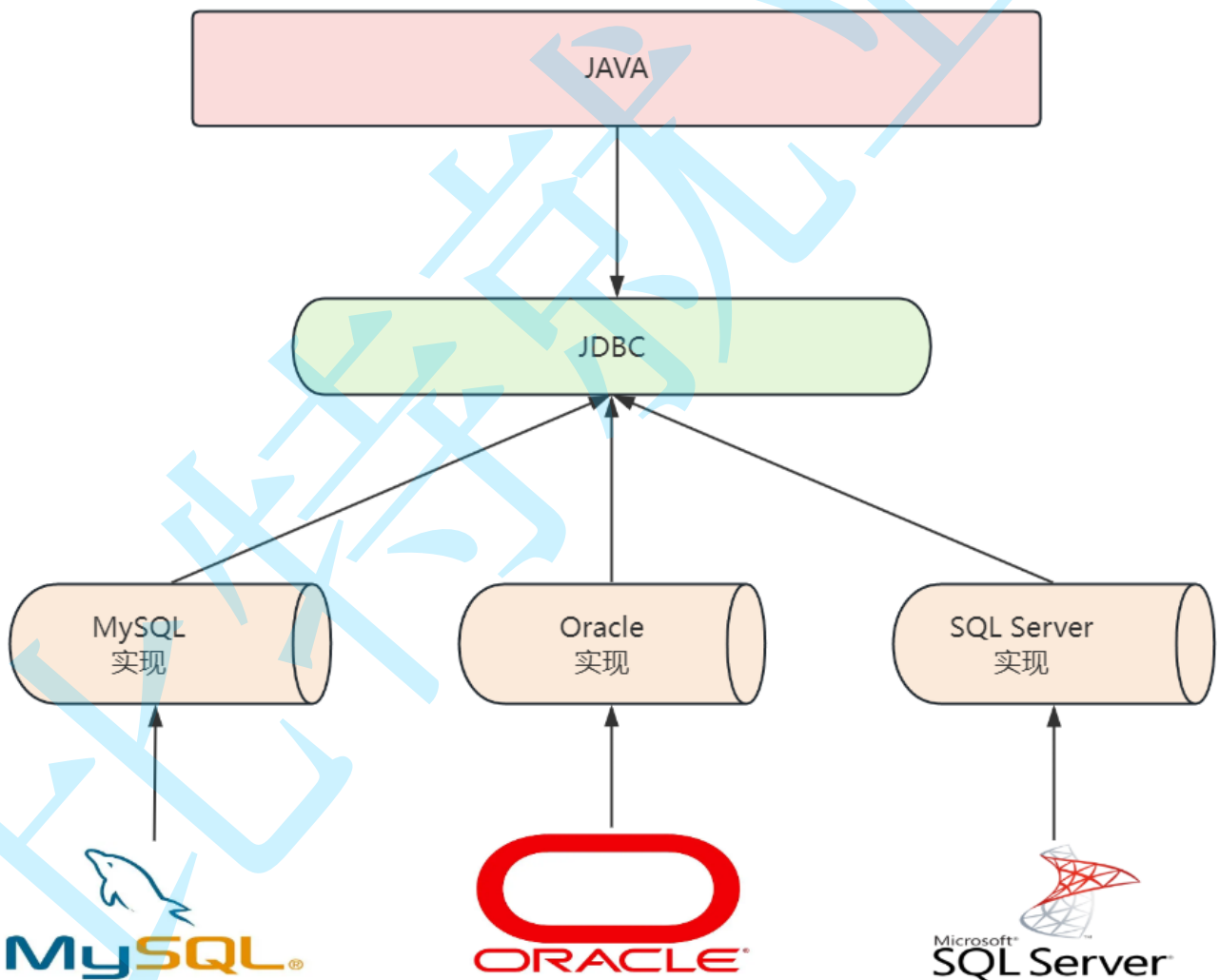
1. 本节目标

掌握JDBC常用接口和类

熟练使用JDBC进行数据库交互

2. 什么是JDBC

JDBC (Java Data Base Connectivity, Java数据库连接) 是Java程序和数据库之间的桥梁, 包含了一套Java定义的用于执行SQL语句的接口, 使开发者能够编写数据库的程序。JDBC 的主要作用是: 与数据库建立连接、发送SQL语句和处理数据库执行结果。



1. JDBC的应用场景

2. JDBC工作原理

- a. JDBC工作原理简洁地概括为：加载驱动、建立连接、创建Statement、执行SQL、处理结果和关闭资源。

3. 为什么要使用JDBC

- 首先回顾一下使用客户端操作数据库的过程，主要分为以下几步：
 - a. 连接到数据库服务
 - b. 发送SQL语句
 - c. 得到返回结果并显示
 - d. 关闭连接
- 同样如果使用程序操作数据库也会经历以上几步，大家应该可以想到，为实现上述步骤，可以编写相应的代码实现数据库连接，发送SQL语句，处理结果并显示，最后关闭连接。
- 但是不同的数据库对于同一个操作不论是协议还是参数都各有不同，如果让程序员自己去实现，那就必须针对不同的数据库进行编码实现，这个工作量和维护成本显然太大。
- Java采取的做法是把以上操作步骤定义了相应的接口，具体的实现交给数据库厂商去做，Java程序员只需要按照需要调用接口中定义的方法即可，这样不论使用什么数据库，都对于Java程序没有任何影响，即便是换一个数据库，也只需要换一下相应厂商的实现依赖。
- JDBC使用过程可以概括为：加载数据库厂商的驱动包、建立连接、创建Statement、执行SQL、处理结果释放资源和关闭连接。

4. 使用JDBC

4.1 创建Maven工程并配置国内镜像

- 修改Maven配置文件，在 `<mirrors></mirrors>` 标签中添加 `mirror` 子节点
- 单独安装的Maven配置文件路径为：**maven安装目录/conf/settings.xml**
- IDEA自带的Maven配置文件路径为：**IDEA安装目录/plugins/maven/lib/maven3/conf/settings.xml**

```
1 <!-- 加入如下mirror节点 使用国内阿里云仓库镜像 开始 -->
2 <mirror>
3   <id>aliyunmaven</id>
4   <mirrorOf>*</mirrorOf>
5   <name>阿里云公共仓库</name>
6   <url>https://maven.aliyun.com/repository/public</url>
7 </mirror>
8 <mirror>
```

```

9   <id>central</id>
10  <mirrorOf>*</mirrorOf>
11  <name>aliyun central</name>
12  <url>https://maven.aliyun.com/repository/central</url>
13 </mirror>
14 <mirror>
15   <id>spring</id>
16   <mirrorOf>*</mirrorOf>
17   <name>aliyun spring</name>
18   <url>https://maven.aliyun.com/repository/spring</url>
19 </mirror>
20 <!-- 加入如下mirror节点 使用国内阿里云仓库镜像 结束-->

```

4.2 获取MySQL驱动包

- 在Maven仓库 mvnrepository.com 搜索MySQL，找到最新版的驱动包



MySQL Connector Java » 8.0.33

MySQL Connector/J is a JDBC Type 4 driver, which means that it is pure Java implementation of the MySQL protocol and does not rely on the MySQL client libraries. This driver supports auto-registration with the Driver Manager, standardized validity checks, categorized SQLExceptions, support for large update counts, support for local and offset date-time variants from the java.time package, support for JDBC-4.x XML processing, support for per connection client information and support for the NCHAR, NVARCHAR ...

Categories	JDBC Drivers
Tags	database sql jdbc driver connector rdbms mysql connection
Date	Apr 18, 2023
Files	pom (2 KB) jar View All
Repositories	Central Loeyae Mulesoft
Ranking	#71 in MvnRepository (See Top Artifacts) #1 in JDBC Drivers
Used By	7,837 artifacts

This artifact was moved to:

[com.mysql » mysql-connector-j](#)

MySQL Connector/J artifacts moved to reverse-DNS compliant Maven 2+ coordinates.

[Maven](#)
[Gradle](#)
[Gradle \(Short\)](#)
[Gradle \(Kotlin\)](#)
[SBT](#)
[Ivy](#)
[Grape](#)
[Leiningen](#)
[Builder](#)

```

<!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>

```

4.3 修改pom.xml文件

在工程中的pom.xml中的 `<dependencies></dependencies>` 标签中添加MySQL依赖，如下所示：

```
1 <dependencies>
2     <!-- MySQL 驱动包 -->
3     <dependency>
4         <groupId>mysql</groupId>
5         <artifactId>mysql-connector-java</artifactId>
6         <version>8.0.33</version>
7     </dependency>
8 </dependencies>
```

4.4 建立数据库连接

- 使用驱动管理类 `DriverManager` 的静态方法获取数据库连接

```
1 // 注册驱动
2 Class.forName("com.mysql.cj.jdbc.Driver");
3
4 // 获取数据库连接
5 Connection connection =
    DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/jobs_info_db?
    characterEncoding" +
6        "utf8&allowPublicKeyRetrieval=true&useSSL=false",
7        "root",
8        "123456");
```

- 通过数据源 `DataSource` 对象获取，推荐在实际开发中应用这种方式

```
1 // 设置数据源的连接串、用户名和密码
2 MysqlDataSource mysqlDataSource = new MysqlDataSource();
3 mysqlDataSource.setURL("jdbc:mysql://127.0.0.1:3306/jobs_info_db?
    characterEncoding" +
4        "utf8&allowPublicKeyRetrieval=true&useSSL=false");
5 mysqlDataSource.setUser("root");
6 mysqlDataSource.setPassword("123456");
7 // 把mysqlDataSource 转换为JDBC的dataSource
8 DataSource dataSource = mysqlDataSource;
```

MySQL数据库连接URL格式：`jdbc:mysql://服务器地址:端口/数据库?参数名=值[&参数名=值]...`

4.5 创建Statement

- Statement是用于执行静态SQL语句并返回执行结果的对象

```
1 // 通过connection获取statement对象
2 Statement statement = connection.createStatement();
```

4.6 执行SQL语句

- 执行select查询时返回的是一个结果集，用ResultSet接收
- 执行insert, update, delete操作时，返回的是受影响的行数，用int类型接收

```
1 // 执行select语句，并接收结果集
2 ResultSet resultSet = statement.executeQuery("select id, name, sno, age,
gender, enroll_date, class_id from student");
3
4 // 执行insert, update, delete语句，并接收受影响的行数
5 int row = statement.executeUpdate("update student set age = 20 where id = 2");
```

4.7 处理结果集

如果返回的是一个结果集，则需要遍历这个集合获取对应列的值，具体代码如下：

```
1 // 遍历结果集获取数据
2 while (resultSet.next()) {
3     // 编号
4     long id = resultSet.getLong("id");
5     // 姓名
6     String name = resultSet.getString("name");
7     // 学号
8     String sno = resultSet.getString("sno");
9     // 年龄
10    int age = resultSet.getInt("age");
11    // 性别
12    byte gender = resultSet.getBytes("gender");
13    // 入学时间
14    Date enrollDate = resultSet.getDate("enroll_date");
15    // 班级编号
16    long classId = resultSet.getLong("class_id");
17    // TODO: 使用获取到的值...
18 }
```

4.8 释放资源

- 在整个数据库访问过程中创建的对象都需要释放，包括：ResultSet, Statement和Connection，后创建的先释放

```
1 // 释放结果集
2 if (resultSet != null) {
3     try {
4         resultSet.close();
5     } catch (SQLException e) {
6         e.printStackTrace();
7     }
8 }
9 // 释放statement
10 if (statement != null) {
11     try {
12         statement.close();
13     } catch (SQLException e) {
14         e.printStackTrace();
15     }
16 }
17 // 关闭连接
18 if (connection != null) {
19     try {
20         connection.close();
21     } catch (SQLException e) {
22         e.printStackTrace();
23     }
24 }
```

5. JDBC常用接口和类

5.1 DriverManager 和DataSource

- DataSource驱动管理类，用于管理JDBC驱动程序，可以从驱动程序中获取数据库连接，始于JDK1.1。
- DataSource数据源是DriverManager的替代方案，始于JDK1.4，是获取数据库连接的首选方法，推荐使用。

5.2 DriverManager 与 DataSource 的区别

- DriverManager和DataSource都可以获取到数据库连接，但它们之间存在着一些区别，主要在于连接的管理方式和资源利用效率

- 连接管理方式不同：
 - DriverManager每次调用getConnection方法都会初始化一个新的连接，使用完成后会关闭真实连接，导致资源浪费
 - DataSource使用了连接池的技术，会在初始化时创建一定数量的数据库连接，这些连接可以重复使用，关闭时并不是真正关闭连接，而是将连接归还给连接池，以供后续使用，有效地提高资源利用率和性能

5.3 Connection 数据库连接

数据库连接(会话)对象，在连接的上下文中执行SQL语句并返回结果

5.4 Statement 对象

5.4.1 Statement

- 用于执行静态SQL语句并返回执行结果，由于只能执行静态语句，所以这里会有一个问题，假设一个语句中需要动态的参数，比如where子句中的条件，那么只能通过字符串拼接的方式组装完成的SQL语句，比如：

```
1 String sql = "select * from student where name = '" + name + "' and class_id = " + classId;
```

- 字符串拼接形式构造SQL语句时，如果不处理参数中的特殊字符就会造成SQL注入，这是一个非常严重的安全性问题。

5.4.2 SQL注入

SQL注入即是指web应用程序对用户输入数据的合法性没有判断或过滤不严，攻击者可以在web应用程序中事先定义好的查询语句的结尾上添加额外的SQL语句，在管理员不知情的情况下实现非法操作，以此来实现欺骗数据库服务器执行非授权的任意查询，从而进一步得到相应的数据信息。

示例：

- 姓名条件**宋江**和班级编号条件**1**是动态的参数

```
1 # 正常的查询，没有符合的结果
2 mysql> select * from student where name = '宋江' and class_id = 1;
3 Empty set (0.01 sec)
```

- 构造恶意参数来执行非法的SQL，把正常的条件值**宋江**替换为 `' or 1 = 1;--`

```

1 # 查询出来了所有的记录，此时学生表中的数据全部都将泄漏
2 mysql> select * from student where name = '' or 1 = 1; -- 宋江' and class_id = 1;
3 +-----+-----+-----+-----+-----+-----+-----+
4 | id | name      | sno      | age | gender | enroll_date | class_id |
5 +-----+-----+-----+-----+-----+-----+-----+
6 | 1 | 唐三藏    | 100001    | 18 | 1      | 1986-09-01  | 1        |
7 | 2 | 孙悟空    | 100002    | 18 | 1      | 1986-09-01  | 1        |
8 | 3 | 猪悟能    | 100003    | 18 | 1      | 1986-09-01  | 1        |
9 | 4 | 沙悟净    | 100004    | 18 | 1      | 1986-09-01  | 1        |
10 | 5 | 宋江      | 200001    | 18 | 1      | 2000-09-01  | 2        |
11 | 6 | 武松      | 200002    | 18 | 1      | 2000-09-01  | 2        |
12 | 7 | 李逵      | 200003    | 18 | 1      | 2000-09-01  | 2        |
13 | 8 | 不想毕业  | 200004    | 18 | 1      | 2000-09-01  | 2        |
14 +-----+-----+-----+-----+-----+-----+-----+
15 8 rows in set (0.01 sec)

```

- 还要以构造其他的非法语句，比如插入、更新和删除等等

```

1 mysql> select * from student where name = '';update student set age=19 where
  id=8; -- 宋江' and class_id = 1
2 Empty set (0.00 sec)
3
4 Query OK, 0 rows affected (0.00 sec)
5 Rows matched: 1  Changed: 0  Warnings: 0
6
7 # 更新成功
8 mysql> select * from student;
9 +-----+-----+-----+-----+-----+-----+-----+
10 | id | name      | sno      | age | gender | enroll_date | class_id |
11 +-----+-----+-----+-----+-----+-----+-----+
12 | 1 | 唐三藏    | 100001    | 18 | 1      | 1986-09-01  | 1        |
13 | 2 | 孙悟空    | 100002    | 18 | 1      | 1986-09-01  | 1        |
14 | 3 | 猪悟能    | 100003    | 18 | 1      | 1986-09-01  | 1        |
15 | 4 | 沙悟净    | 100004    | 18 | 1      | 1986-09-01  | 1        |
16 | 5 | 宋江      | 200001    | 18 | 1      | 2000-09-01  | 2        |
17 | 6 | 武松      | 200002    | 18 | 1      | 2000-09-01  | 2        |
18 | 7 | 李逵      | 200003    | 18 | 1      | 2000-09-01  | 2        |
19 | 8 | 不想毕业  | 200004    | 19 | 1      | 2000-09-01  | 2        |
20 +-----+-----+-----+-----+-----+-----+-----+
21 8 rows in set (0.00 sec)

```

同学们不要去做没有授权的安全测试

5.4.3 PreparedStatement

预编译SQL语句对象，SQL语句被预编译并存储在PreparedStatement对象中，可以使用该对象多次执行SQL语句，同时可以解决SQL注入问题。

示例：通过Connection对象获取到PreparedStatement对象，需要传入SQL模板，动态参数用占位符 `?` 表示

```
1 // 获取了个处理SQL的PreparedStatement对象
2 PreparedStatement preparedStatement = connection.prepareStatement("select id,
    name, sno, age, gender, enroll_date, class_id from student where name = ? and
    class_id = ?");
3
```

- 为动态参数设置真实值，下标从1开始

```
1 // 用真实值去替换占位符
2 preparedStatement.setString(1, "宋江");
3 preparedStatement.setLong(2, 2);
```

- 执行SQL语句

```
1 // select 操作
2 ResultSet resultSet = statement.executeQuery();
3
4 // insert, update, delete操作
5 int result = statement.executeUpdate();
```

5.4.4 CallableStatement

用于执行SQL存储过程的接口。这里不做讨论

5.4.5 executeQuery()

执行结果返回的是一个结果集，通常用于select 操作

5.4.6 executeUpdate()

执行结果返回的是一个整形，通常用于insert, update, delete操作

5.5 ResultSet结果集

- 是一个查询结果集的数据表，通常由执行查询操作的语句生成。

- ResultSet对象维护了一个指向当前数据行的游标，最初游标位于第一行之前，调用next方法将游标移动到下一行，当ResultSet中没有更多的数据行时返回false，所以可以在while循环中使用它来遍历结果集。
- ResultSet接口提供了getter方法(getBoolean、getLong等)，用于从当前行检索列值，可以使用列的索引号或列的名称来检索值。一般来说，使用列索引会更有效率，索引编号从1开始，按照从左到右的顺序读取。

6. 示例

- 查询学生编号为1的学生基本信息

```
1 package org.bitejiuyeyeke;
2
3 import com.mysql.cj.jdbc.MySQLDataSource;
4
5 import javax.sql.DataSource;
6 import java.sql.*;
7 import java.text.MessageFormat;
8
9 /**
10  * @Author 比特就业课
11  */
12
13 public class Demo_Database {
14     public static void main(String[] args) {
15         // 定义数据源
16         DataSource dataSource = null;
17         // 定义连接对象
18         Connection connection = null;
19         // 定义预编译对象
20         PreparedStatement statement = null;
21         // 定义结果集对象
22         ResultSet resultSet = null;
23
24         // 设置数据源参数
25         MySQLDataSource mysqlDataSource = new MySQLDataSource();
26         mysqlDataSource.setURL("jdbc:mysql://127.0.0.1:3306/java01?
characterEncoding=utf8&allowPublicKeyRetrieval=true&useSSL=false");
27         mysqlDataSource.setUser("root");
28         mysqlDataSource.setPassword("123456");
29         // 转为JDBC数据源
30         dataSource = mysqlDataSource;
31
32         // 获取连接
```

```
33     try {
34         connection = dataSource.getConnection();
35         // 获取了个处理SQL的PreparedStatement对象
36         statement = connection.prepareStatement("select id, name, sno,
age, gender, " +
37             "enroll_date, class_id from student where id = ?");
38         // 用真实值去替换占位符
39         statement.setLong(1, 1);
40
41         resultSet = statement.executeQuery();
42         // 处理结果集, 由于查询的是一条记录, 这里用if即可
43         if (resultSet.next()) {
44             long id = resultSet.getLong("id");
45             String name = resultSet.getString("name");
46             String sno = resultSet.getString("sno");
47             int age = resultSet.getInt("age");
48             byte gender = resultSet.getBytes("gender");
49             Date enrollDate = resultSet.getDate("enroll_date");
50             long classId = resultSet.getLong("class_id");
51             // 打印结果
52             System.out.println(MessageFormat.format("{0}, {1}, {2}, {3},
{4}, {5}, {6}", id, name, sno, age, gender,
53                 enrollDate, classId));
54         }
55     } catch (SQLException e) {
56         e.printStackTrace();
57     } finally {
58         // 在finally中释放所有资源
59         // 关闭结果集
60         if (resultSet != null) {
61             try {
62                 resultSet.close();
63             } catch (SQLException e) {
64                 e.printStackTrace();
65             }
66         }
67         // 关闭预编译对象
68         if (statement != null) {
69             try {
70                 statement.close();
71             } catch (SQLException e) {
72                 e.printStackTrace();
73             }
74         }
75         // 关闭数据库连接
76         if (connection != null) {
77             try {
```

```
78         connection.close();
79     } catch (SQLException e) {
80         e.printStackTrace();
81     }
82 }
83 }
84 }
85 }
```

7. 作业

1. 使用JDBC对学生表进行增删除改查操作
2. 查询所有同学个人信息和所在班级